SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1741**

CO1-AT2-Constraint-Based Problem Solving

Register Number	**192412204**
Name	**S.Arshad Ali**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 20%

QUESTIONS:3 **Total Marks:30**

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift

   ii. D2 must work before D3 (Morning < Afternoon < Night)

   iii. Only one doctor per shift

   iv. D3 cannot work Morning

   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost**.**

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.
   The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

   Constraints:

   - The robot can move up, down, left, right

- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:
a) Analyze all the given constraints and explain how they affect the robot's decision-making process.
b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
c) Demonstrate how the robot applies AI concepts such as:
- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Criteria	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	**Max Marks**
Problem Understanding	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	6
Constraint Analysis	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	6
Solution Development	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	7.5

Application of Concepts	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	6
Justification	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	4.5

ANSWERS:

1) A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

I. D1 cannot work Night shift

II. D2 must work before D3 (Morning < Afternoon < Night)

III. Only one doctor per shift

IV. D3 cannot work Morning

V. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

ANS:

1. Aim

To assign three doctors \(D1, D2, D3\) to three shifts — Morning, Afternoon, and Night — using Backtracking Search, while satisfying all the given constraints.

2. Problem Formulation

Variables: \(D1, D2, D3\)

Domain: Each doctor can be assigned one of the following shifts:

\[\text{Domain}=\{M,A,N\} \]

Where:

- \(M\) = Morning
- \(A\) = Afternoon
- \(N\) = Night

Constraints:

1. \(D1 \ne N\): D1 cannot work the Night shift.
2. \(D2 < D3\): D2 must work an earlier shift than D3.
3. All doctors must have different shifts.
4. \(D3 \ne M\): D3 cannot work the Morning shift.
5. Each doctor must be assigned exactly one shift.

3. Backtracking Search Algorithm

Backtracking Search assigns a shift to one doctor at a time. After each assignment, it checks whether the constraints are satisfied. If a constraint is violated, the algorithm rejects that assignment and tries another shift.

Pseudocode

BACKTRACK(doctor, assignment):

```
IF all doctors are assigned:
   RETURN assignment

FOR each shift in {Morning, Afternoon, Night}:

   Assign shift to doctor

   IF all constraints are satisfied:
      result = BACKTRACK(next doctor, assignment)

      IF result is successful:
         RETURN result

   Remove the assignment

RETURN FAILURE
```

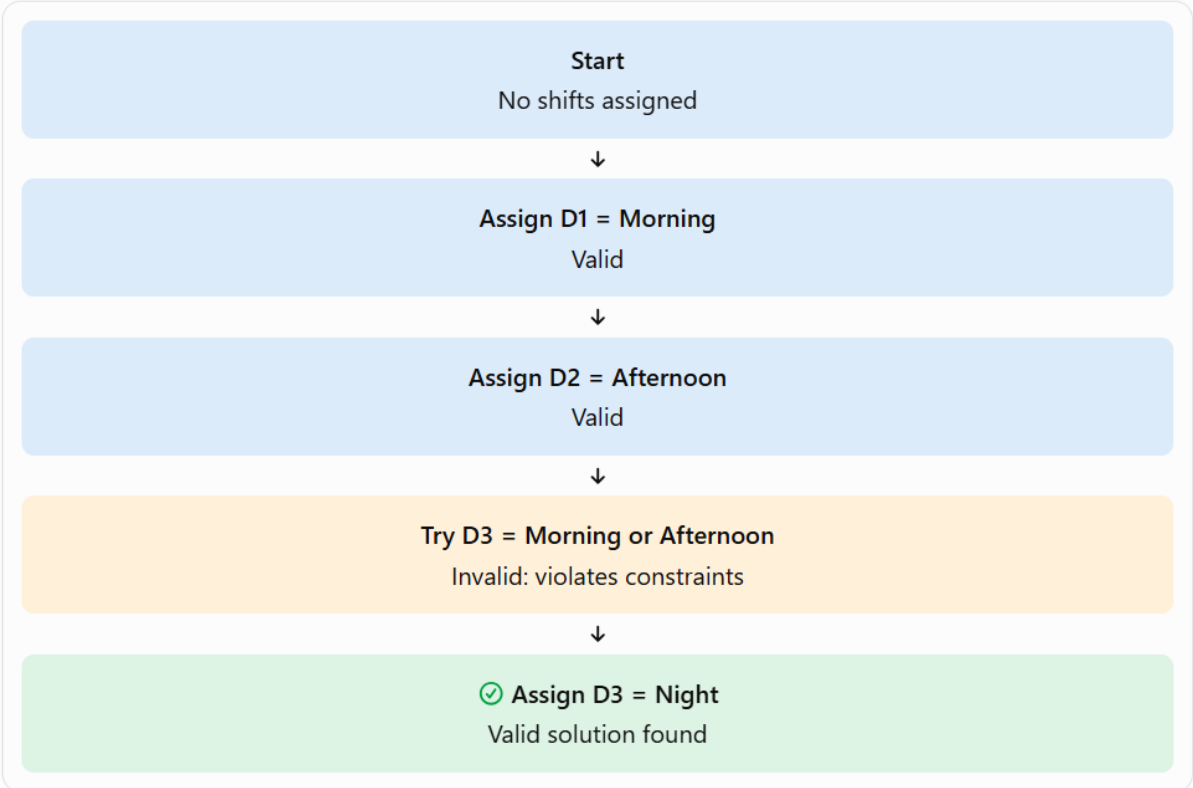
4. Step-by-Step Execution with Constraint Checks

We assign shifts in the order \(D1 \rightarrow D2 \rightarrow D3\).

Step	Assignment	Constraint check	Decision
1	\(D1 = M\)	D1 is not assigned Night.	Accept
2	\(D2 = M\)	Two doctors cannot share a shift.	Reject
3	\(D2 = A\)	D2 has a different shift from D1.	Accept
4	\(D3 = M\)	D3 cannot work Morning; also conflicts with D1.	Reject
5	\(D3 = A\)	D3 would share the Afternoon shift with D2.	Reject
6	\(D3 = N\)	D3 is not Morning, and D2 works before D3.	Accept

All constraints are now satisfied, so the algorithm terminates successfully.

5. Backtracking Search Tree



6. Final Valid Schedule

Verification of Constraints

- Constraint 1: D1 works Morning, not Night. ✓
- Constraint 2: D2 works Afternoon, before D3, who works Night. ✓
- Constraint 3: Each shift has exactly one doctor. ✓
- Constraint 4: D3 works Night, not Morning. ✓
- Constraint 5: All three doctors are assigned exactly one shift. ✓

7. Conclusion

Backtracking Search successfully finds a valid doctor–shift assignment by assigning shifts one at a time, checking the constraints, and rejecting invalid choices. The final schedule is D1 – Morning, D2 – Afternoon, and D3 – Night. This satisfies all the given constraints.

2) A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

I. **Movement allowed: Up, Down, Left, Right**
II. **Cost of each move = 1**
III. **Cannot pass through obstacles**
IV. **Use Manhattan Distance heuristic**

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

ANS:

1. Aim

To find the optimal path from Start (S) to Goal (G) in a 5×5 grid using search algorithms, avoiding obstacles and calculating the total path cost.

2. Given Data

- Grid size: 5×5
- Start position: S
- Goal position: G
- Allowed movements: Up, Down, Left, Right
- Cost of each move: 1
- Obstacles: Cannot pass through blocked cells.
- Heuristic: Manhattan distance.

Since the question does not specify the exact positions, the following example grid is used for demonstration.

4. Manhattan Distance Heuristic

The Manhattan distance estimates the distance between the current node and the goal.

$$h(n)=|r_n-r_G|+|c_n-c_G|$$

For the Start position $S=(1,1)$ and Goal $G=(5,5)$:

\[h(S)=|1-5|+|1-5|=8 \]

Each horizontal or vertical move costs 1. Obstacles may increase the actual travel cost, but Manhattan distance never overestimates it in this grid.

5. BFS Search Algorithm:

```
BFS(Start, Goal):

    Create an empty queue Q
    Insert Start into Q
    Mark Start as visited
    Store parent[Start] = NULL

    WHILE Q is not empty:

        current = REMOVE_FRONT(Q)

        IF current = Goal:
            RETURN reconstruct_path(parent, Goal)

        FOR each neighbor in (Right, Down, Left, Up):

            IF neighbor is inside the grid
               AND neighbor is not an obstacle
               AND neighbor is not visited:

                Mark neighbor as visited
                parent[neighbor] = current
```

INSERT neighbor into Q

RETURN FAILURE

6. Step-by-Step Node Expansion and Queue Updates

Coordinates are written as \((row,column)\). The queue is shown from front to rear. Neighbors are checked in the order Right, Down, Left, Up.

Step	Expanded node	Newly discovered nodes	Queue after expansion
1	\((1,1)\) S	\((1,2)\)	\((1,2)\)
2	\((1,2)\)	\((1,3)\)	\((1,3)\)
3	\((1,3)\)	\((2,3)\)	\((2,3)\)
4	\((2,3)\)	\((3,3)\)	\((3,3)\)
5	\((3,3)\)	\((4,3),(3,2)\)	\((4,3),(3,2)\)
6	\((4,3)\)	\((4,4)\)	\((3,2),(4,4)\)
7	\((3,2)\)	\((3,1)\)	\((4,4),(3,1)\)
8	\((4,4)\)	\((4,5),(5,4)\)	\((3,1),(4,5),(5,4)\)
9	\((3,1)\)	None	\((4,5),(5,4)\)
10	\((4,5)\)	\((5,5)\) G, \((3,5)\)	\((5,4),(5,5),(3,5)\)
11	\((5,4)\)	None	\((5,5),(3,5)\)
12	\((5,5)\) G	Goal reached	Stop

The search terminates when the goal is removed from the queue. Since every move has equal cost, BFS guarantees the shortest path in terms of the number of moves.

8. Important Note About the Heuristic

The question mentions both BFS and Manhattan distance. Strictly speaking, BFS does not use a heuristic or a priority queue; it uses a FIFO queue. If the intention is to use Manhattan distance to guide the search, the appropriate algorithm is A*, which evaluates:

\[f(n)=g(n)+h(n) \]

where \(g(n)\) is the cost from Start to the current node, and \(h(n)\) is the Manhattan distance to Goal.

9. Conclusion

Breadth-First Search finds the optimal path from \(S\) to \(G\) while avoiding all obstacles. For the example grid used here, the robot travels 8 moves with a minimum total cost of 8. Manhattan distance can additionally be used with A* search to guide the robot toward the goal

3)

3) An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- **The robot can move up, down, left, right**
- **Some cells are blocked (obstacles) and cannot be traversed**
- **The robot has limited sensing capability (can only observe adjacent cells)**
- **Each movement has a cost of 1, but entering risky zones has an additional cost of 2**
- **The robot must minimize total cost while ensuring safe navigation**
- **The robot must update its knowledge dynamically (online search scenario)**

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.

c) Demonstrate how the robot applies AI concepts such as:

- **Intelligent agents**
- **Rationality**
- **Environment type**

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

ANS:

Introduction

An autonomous rescue robot is an intelligent agent designed to locate and rescue survivors in disaster-affected buildings. The building is represented as a grid of rooms containing obstacles and risky zones. The robot must travel from the starting position \(S\) to the survivor's location \(G\) while minimizing the total cost and ensuring safe navigation.

Since the robot can observe only adjacent cells and must update its knowledge as it moves, this problem is an example of online search in a partially observable environment.

(a) Analysis of Constraints and Their Effects

Constraint	**Effect on robot decision-making**
Movement in four directions	The robot can move up, down, left, or right to reach neighboring cells.
Obstacles	Blocked cells must be avoided when planning a route.
Limited sensing	The robot knows only the condition of adjacent cells and must explore to discover unknown areas.
Movement cost = 1	Every normal movement adds 1 to the total cost.
Risky-zone cost = 2	Entering a risky cell costs \(1+2=3\), so safer routes are preferred

extra	when possible.
Minimum total cost	The robot must compare alternative routes and select the least-cost safe path.
Dynamic knowledge	The robot updates its internal map after sensing new cells and discovering obstacles or hazards.

The total path cost is calculated as

\[C(\text{path})=\sum_{i=1}^{n} \left(1+2R_i\right) \]

where \(R_i=1\) if the entered cell is risky, and \(R_i=0\) otherwise.

Thus, a normal cell costs 1, while a risky cell costs 3.

(b) Solution Using Uniform Cost Search and an Online Search Agent

Uniform Cost Search (UCS) is suitable for finding a minimum-cost route when movement costs are non-negative. An online search agent can complement UCS by sensing the surroundings, updating its map, and replanning when new information becomes available.

Step 1: Problem formulation

- Initial state: Robot at starting position \(S\).
- Goal state: Survivor reached at \(G\).
- Actions: Move up, down, left, or right.
- Transition model: Moving to a valid adjacent cell changes the robot's position.
- Path cost: 1 for a normal cell and 3 for a risky cell entered.
- Goal test: Check whether the robot has reached \(G\).

Step 2: Search algorithm

```
ONLINE_RESCUE_AGENT(S, G)


  position ← S

  known_map ← initial observations


  WHILE position ≠ G:


    Observe adjacent cells

    Update known_map
```

```
    Mark blocked cells as unavailable

    Record normal and risky cells


    path ← UCS(
        position,
        G,
        known_map,
        movement_cost = 1,
        risky_cell_cost = 3
    )


    IF no safe known path exists:
        Explore a safe, reachable unknown cell
        Update known_map
    ELSE:
        Move one step along the selected path


    Update position and observations


RETURN "Survivor reached"
```

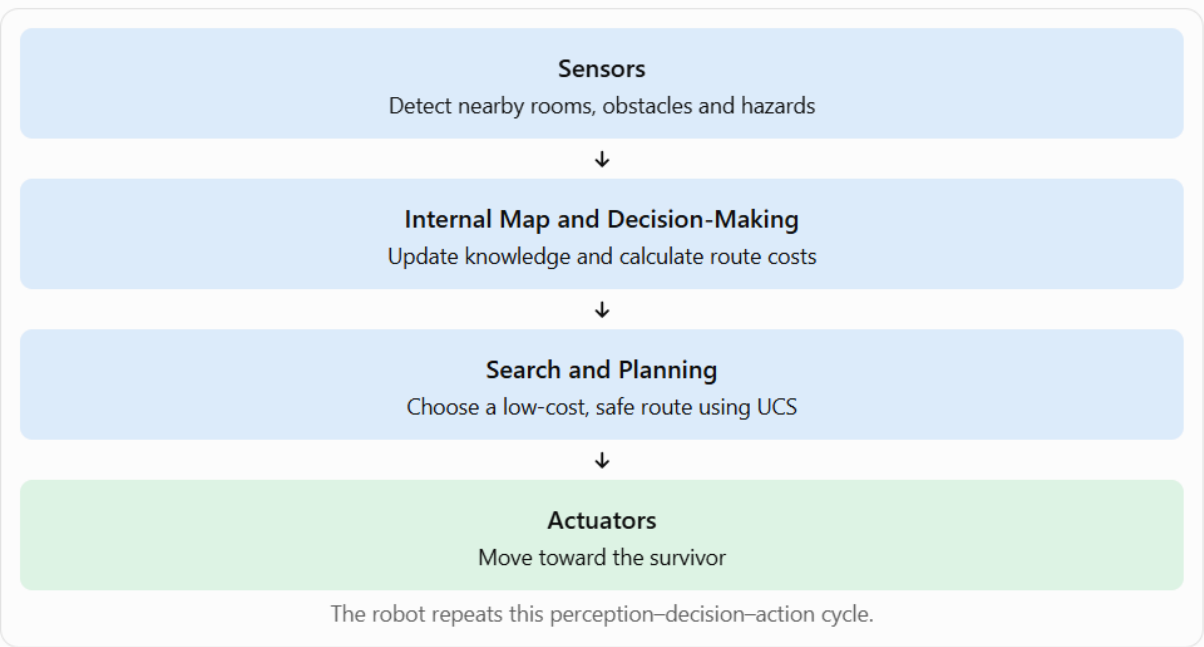
Step 3: Working procedure

1. Start at \(S\) and observe the four adjacent cells.
2. Identify blocked cells and record known risky zones.
3. Generate reachable neighboring positions.
4. Calculate the cumulative cost of each candidate route.
5. Use UCS to select the lowest-cost route through the currently known map.
6. Move one step, observe the surroundings, and update the map.
7. If a newly discovered obstacle blocks the planned route, run the search again.
8. Continue until the robot reaches \(G\) safely.

(c) Application of Artificial Intelligence Concepts

1. Intelligent agent

An intelligent agent perceives its environment using sensors and performs actions to achieve a goal.



2. Rationality

A rational agent selects the action expected to achieve the best result based on its available information.

The rescue robot demonstrates rationality by:

- Avoiding known obstacles.
- Preferring lower-cost routes.
- Avoiding risky zones when a suitable safer alternative exists.
- Updating its decisions when new information is obtained.
- Prioritizing survivor access and safety over unnecessary movement.

Rationality does not mean the robot always knows the perfect route. It means it makes the best decision based on its current knowledge and performance criteria.

(d) Justification for the Chosen Approach

The combination of UCS and an online search agent is suitable for the following reasons:

1. Minimum-cost path: UCS expands the lowest cumulative-cost node first, accounting for the extra cost of risky zones.
2. Obstacle avoidance: Known blocked cells are excluded from the search.
3. Adaptability: The online agent can revise its map and replan after discovering new obstacles or hazards.

4. Partial observability: The robot does not need a complete map before beginning exploration.
5. Safety-aware planning: Risk penalties help the robot prefer safer routes, although strict safety requirements should be enforced by excluding unacceptable routes.
6. Optimality: UCS finds the least-cost route on a fully known graph with non-negative costs. Replanning allows the robot to respond to newly discovered information, but it does not guarantee a globally optimal route when the environment is initially unknown.

Conclusion

An autonomous rescue robot can be designed as a rational, goal-directed intelligent agent that combines Uniform Cost Search with online exploration. It senses adjacent cells, updates its internal map, avoids obstacles, considers the additional cost of risky zones, and replans when necessary. This approach supports safe and cost-effective navigation toward survivors in partially observable disaster environments.